

트랜스포머

7.1. 트랜스포머 개요

- 2017년 코넬 대학의 아시시 바스와니 등의 연구 그룹이 발표한 'Attention is All You Need' 논문을 통해 소개된 신경망 아키텍처
- 기존 순환 신경망과 같은 순차적 방식이 아닌 병렬로 입력 시퀀스를 처리
 - ⇒ 긴 시퀀스의 경우, 트랜스포머 모델을 RNN보다 훨씬 더 빠르고 효율적으로 처리
 - ⇒ **셀프 어텐션(Self-Attention)** 기반
- **오토 인코딩(Auto-Encoding)**, **자기 회귀(Auto-Regressive)** 방식 또는 두 개의 조합으로 학습

셀프 어텐션(Self-Attention)

- 순차 처리(Sequention Processing)나 반복 연결(Recurrent Connections)에 의존하지 않고 입력 토큰 간의 관계를 직접 처리, 이해할 수 있도록 함
- 모델이 재귀나 합성곱 연산 없이 입력 토큰 간의 관계를 직접 모델링할 수 있음

오토 인코딩(Auti-Encoding)

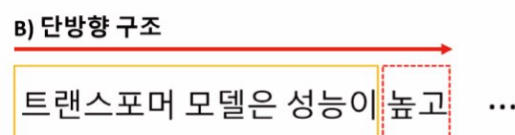
- 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업(Task)를 수행
- 예측되는 토큰의 양옆에 있는 토큰들을 참조
 - ⇒ **양방향 구조**
 - ⇒ **인코더(Encoder)**
- 구조



- 입력: '트랜스포머 모델은 성능이', '다양한 구조로 활용된다'
- 출력: '높고'

자기 회귀 방식(Auto-Regressive)

- 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞추는 작업을 수행
- 예측되는 토큰의 왼쪽에 있는 토큰들만 참조
 - ⇒ **단방향 구조**
 - ⇒ **디코더(Decoder)**
- 구조



- 입력: '트랜스포머 모델은 성능이'
- 출력: '높고'

모델 특징

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

- ELECTRA: 인코더에 생성적 적대 신경망의 판별기를 활용한 모델
- T5, BART: 다양한 자연어 처리 작업에 대한 학습을 수행

사용

- 트랜스포머 모델의 학습은 대용량 데이터셋에서 매우 효율적
⇒ 데이터 양이 많은 기계 번역과 같은 작업에 적합
- 언어 모델링, 텍스트 분류에서 매우 효과적
- 광범위한 자연어 처리 작업에서 높은 효율을 보임
- 기계 번역, 언어 모델링, 텍스트 요약 같은 장기적은 종속성을 포함하는 작업에서 주로 사용
- 자연어 처리 분야에서 널리 사용되고 영향력이 큼

7.2. 트랜스포머 개념

- 딥러닝 모델 중 하나
- 기계 번역, 챗봇, 음성 인식 등 다양한 자연어 처리 분야에서 많은 성과를 내는 모델
- RNN, CNN과 달리 어텐션 메커니즘만을 사용하여 시퀀스 임베딩을 표현

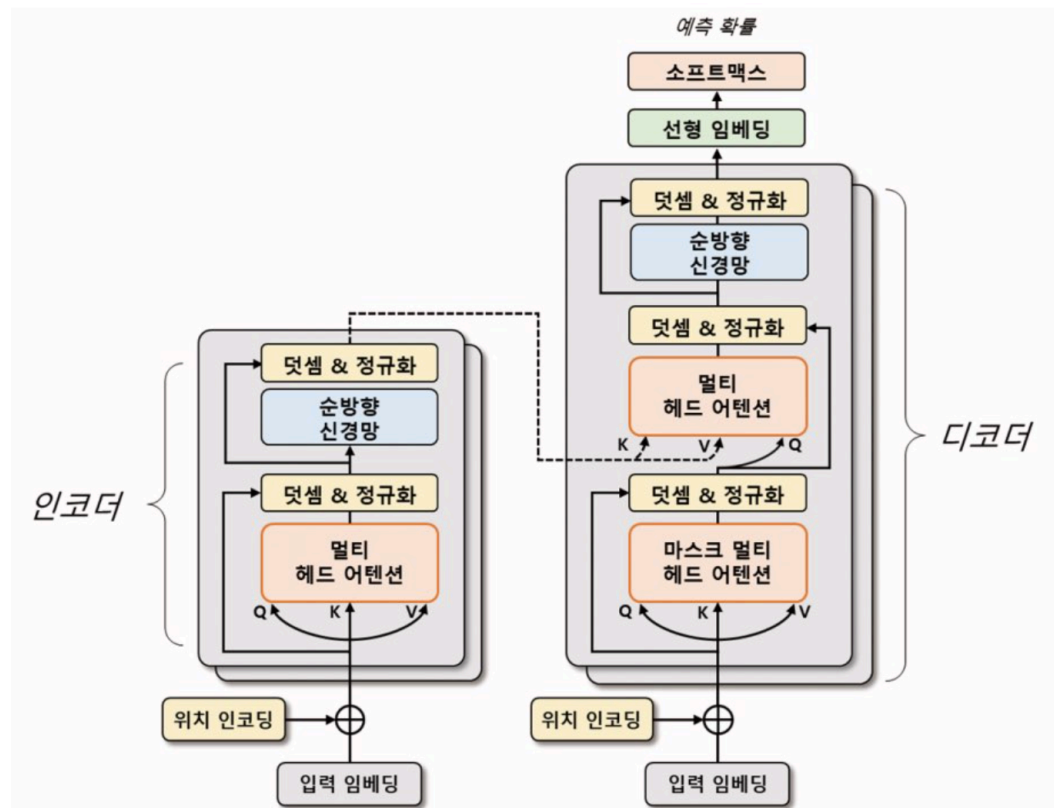
7.2.1. 어텐션 메커니즘

- 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 적절한 출력을 생성
- 인코더: 입력 시퀀스를 임베딩 ⇒ 고차원 벡터로 변환
- 디코더: 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성
- 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중
⇒ 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악
⇒ 번역, 요약문 생성과 관련된 작업 등을 수행

7.2.2. 장점

- RNN 기반 모델보다 학습 속도가 빠르고, 병렬 처리가 가능
⇒ 대규모 데이터셋에서 높은 성능을 보임
- 임베딩 과정에서 문장의 전체 정보를 고려
⇒ 문장의 길이가 길어져도 성능이 유지

7.2.3. 구조



- 인코더와 디코더는 두 부분으로 구성
- 각각 n개의 **트랜스포머 블록(멀티 헤드 어텐션 + 순방향 신경망)**으로 구성

멀티 헤드 어텐션

- 입력 시퀀스에서 쿼리(Q), 키(K), 값(V) 벡터를 정의
→ 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
- 쿼리와 각 키의 유사도를 계산
→ 해당 유사도를 가중치로 사용하여 값 벡터를 합산
- 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체
⇒ **입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신**

순방향 신경망

- 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용
- 여러 개의 선형 계층으로 구성
- 입력 벡터에 가중치를 곱하고, 편향을 더하며, 활성화 함수를 적용
- 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신

입력 시퀀스 데이터의 처리

- 소스(source)와 타겟(target) 데이터로 나눠 처리
- 예시) 영어를 한글로 번역하는 경우
 - 타겟 데이터: 생성하는 언어인 한글
 - 소스 데이터: 참조하는 언어인 영어

인코더

- 소스 시퀀스 데이터를 위치 인코딩(Positional Encoding)된 입력 임베딩으로 표현 → 트랜스포머 블록의 출력 벡터 생성
- 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성

디코더

- 인코더와 유사하게 트랜스포머 블록으로 구성
- 마스크 멀티 헤드 어텐션(Masked Multi-Head Attention) 사용
⇒ 타겟 시퀀스 데이터를 순차적으로 생성

- 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조
- 최종적으로 생성된 디코더 출력 벡터
 - ⇒ 선형 임베딩으로 재표현
 - ⇒ 이미지/자연어 모델에 활용

7.3. 입력 임베딩과 위치 인코딩

7.3.1. 위치 인코딩

위치 인코딩이 필요한 이유

- 입력 시퀀스의 각 단어: 임베딩 처리되어 벡터 형태로 변환
- RNN과 달리 입력 시퀀스를 병렬 구조로 처리
 - ⇒ 단어의 순서 정보를 제공하지 않음
- **위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영**
 - ⇒ 위치 인코딩 방식 사용

위치 인코딩 개념

- 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
- 각 단어의 위치 정보를 나타내는 벡터를 더해 임베딩 벡터에 위치 정보를 반영

위치 인코딩 벡터 생성

- 각 토큰의 위치를 각도로 표현해 $\sin$ 함수와 $\cos$ 함수로 위치 인코딩 벡터를 계산
 - ⇒ 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터 생성
- 토큰의 위치마다 동일한 임베딩 벡터를 동일한 임베딩 벡터를 사용하지 않음
 - ⇒ 각 토큰의 위치 정보를 모델이 학습
- 모델은 단어의 순서 정보를 학습할 수 있음

7.3.2. 위치 인코딩 계산 과정

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

        pe = torch.zeros(max_len, 1, d_model) # 텐서 차원: [50, 1, 128] = [최대 시퀀스, 1, 입력 임베딩의 차원]
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe) # 모델이 매개변수를 갱신하지 않도록 설정

    def forward(self, x): # 산출된 위치 인코딩 + 입력 임베딩 → 드롭아웃 적용
        x = x + self.pe[: x.size(0)] # 산출된 위치 인코딩 + 입력 임베딩
        return self.dropout(x) # 드롭아웃 적용

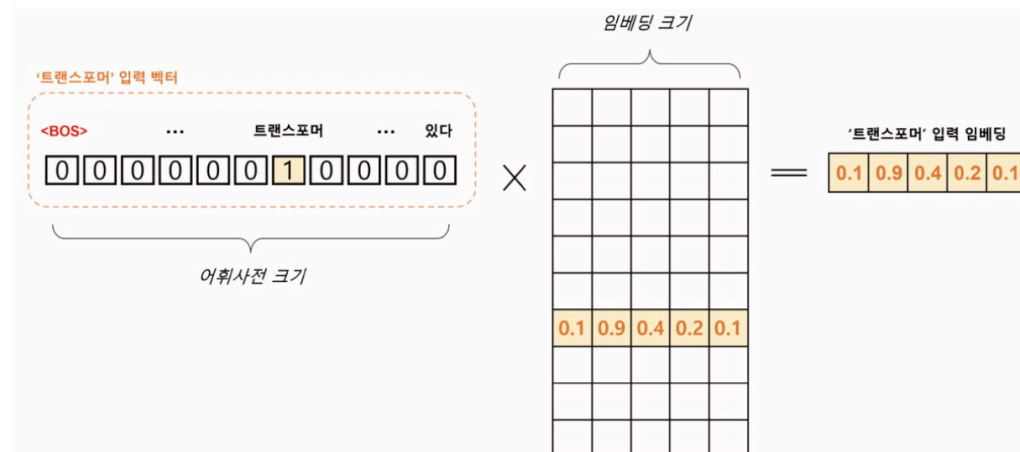
encoding = PositionalEncoding(d_model=128, max_len=50)
```

- 입력: 임베딩 차원(`d_model`), 최대 시퀀스(`max_len`)

- 모델이 이전에 본 적이 없는 단어를 처리할 때 사용
- PAD(Padding)
 - 모든 문장을 일정한 길이로 맞추기 위해 사용
 - 짧은 문장의 빈 공간을 채울 때 사용

2) 원-핫 인코딩(입력 임베딩 변환)

- 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현

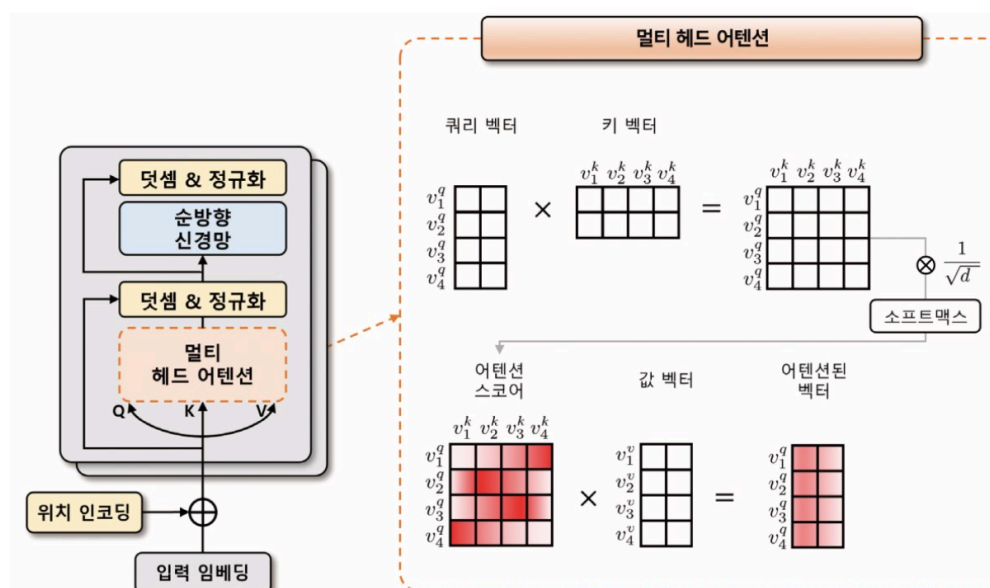


- 입력 임베딩으로 변환되는 과정은 Word2Vec과 동일
 - V: 어휘 사전 크기
 - d: 입력 임베딩 차원
 - '트랜스포머'의 원-핫 벡터 크기: [1, V]
 - ⇒ 원-핫 벡터는 임베딩 행렬 [V, d]에 의해 [1, d] 크기의 벡터로 변환
- N개의 문장이 최대 S개의 토큰 길이를 가질 때
 - [N, S, V] 크기의 원-핫 벡터 텐서 → [N, S, d] 크기의 임베딩 텐서로 변환
- 임베딩 텐서는 입력으로 사용, 트랜스포머의 모든 계층에서 공유되는 텐서로 사용

7.5. 트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행
- 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성
 - ⇒ 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달
- 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더함
- 산출된 인코더 계층의 출력은 디코더 계층으로 전달

7.5.1. 인코더 블록의 연산 과정



- 입력
 - 위치 인코딩이 적용된 소스 데이터의 입력 임베딩
- 멀티 헤드 어텐션
- 닷셈 & 정규화
- 순방향 신경망

1) 입력

- 입력 텐서 차원 [N, s, d]

2) 임베딩 벡터 생성

- 선형 변환을 통해 3개의 임베딩 벡터를 생성
: 쿼리(Q), 키(K), 값(V) 벡터
- 쿼리 벡터(v^q)
 - 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터
 - 인코더의 각 시점마다 생성
 - 현재 시점에서 질문이 되는 벡터
 - 쿼리 벡터를 기준으로 다른 시점의 정보를 참조
- 키 벡터(v^k)
 - 쿼리 벡터와 비교되는 대상
 - 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
 - 인코더의 각 시점에서 생성
- 값 벡터(v^v)
 - 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할
- 쿼리, 키, 값 벡터는 초기에 비슷한 값을 가지지만, 모델 학습을 통해 각 벡터는 의도한 의미의 값을 갖게 됨

3) 셀프 어텐션

- 쿼리와 키 벡터의 연관성: 내적 연산으로 [N, S, S] 어텐션 스코어 맵을 사용
- 쿼리, 키, 값을 사용한 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

1. 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구함
 2. 스코어 값에 $\sqrt{d}$ (벡터 차원의 제곱근)을 나눠 보정
⇒ 벡터 차원이 커질 때 스코어값이 같이 커지는 문제를 완화
 3. 보정된 어텐션 스코어를 소프트맥스 함수를 이용하여 확률적으로 재표현
 4. 값 벡터와 내적하여 셀프 어텐션된 벡터를 생성
- 이러한 과정을 반복해 셀프 어텐션된 스코어 맵을 생성

4) 멀티 헤드

- 셀프 어텐션을 여러 번 수행해 여러 개의 헤드 생성
- 각각의 헤드가 독립적으로 어텐션을 수행하고 그 결과를 합침
- 예시) [N, S, d] 텐서에 k개의 셀프 어텐션 벡터를 생성
 - 헤드에 대한 차원 축을 생성해 [N, k, S, d/k] 텐서 형태를 구성
 - k개의 셀프 어텐션된 [N, S, d/k] 텐서를 의미

- 생성된 k개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 병합되어 [N, S, d] 형태로 출력

5) 닷셈 & 정규화

- 멀티 헤드 어텐션을 통과하기 이전의 입력값 [N, S, d] 텐서 + 통과한 이후의 출력값 [N, S, d] 텐서
⇒ 학습 시 발생하는 기울기 소실을 완화
- 이 값에 임베딩 차원 축으로 정규화하는 계층 정규화를 적용

6) 순방향 신경망

- 두 가지 방법
 - 인공 신경망: 선형 임베딩 + ReLU
 - 1차원 합성곱
- 이어서 다시 닷셈 & 정규화 과정이 수행

7) 다른 트랜스포머 인코더 블록

- 트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성
- 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달
- 인코더 블록을 통과하면서 점차 입력 시퀀스의 정보가 추상화

8) 마지막 인코더 블록

- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용
- 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용

7.5.3. 텐서의 변환 과정

1) 입력 임베딩

- 형태: $X \in \mathbb{R}[N, S, d]$
 - N: 배치 크기
 - S: 시퀀스 길이
 - d: 모델 차원(입력 임베딩 차원)

2) 선형 투영 (Linear Projections)

- 세 개의 학습 가능한 가중치 행렬을 통해 Q, K, V를 생성

$$W^Q, W^K, W^V \in \mathbb{R}[d, d]$$

$$b^Q, b^K, b^V \in \mathbb{R}[d]$$

- 투영 결과

$$Q = X \cdot W^Q + b^Q \Rightarrow [N, S, d]$$

$$K = X \cdot W^K + b^K \Rightarrow [N, S, d]$$

$$V = X \cdot W^V + b^V \Rightarrow [N, S, d]$$

3) 헤드 분할 (Split into Heads)

- d를 k개의 헤드로 나눈다고 하면,

$$\text{head_dim} = d / k$$

- Q, K, V 각각을 reshape + transpose

$$[N, S, d] \rightarrow \text{reshape} \rightarrow [N, S, k, \text{head_dim}]$$

→ transpose → [N, k, S, head_dim]

- 결과

$Q_split, K_split, V_split \in \mathbb{R}[N, k, S, head_dim]$

4) 스케일드 닷-프로덕트 어텐션 (Scaled Dot-Product Attention)

- 각 헤드 i에 대해

1. 유사도 계산

$scores_i = Q_split[i] \cdot K_split[i]^T \Rightarrow [N, S, S]$
 $scores_i = scores_i / \sqrt{head_dim}$

2. 소프트맥스

$A_i = \text{Softmax}(scores_i) \Rightarrow [N, S, S]$

3. 가중합

$head_i = A_i \cdot V_split[i] \Rightarrow [N, S, head_dim]$

- 병렬 처리된 k개 헤드를 모으면:

$heads \in \mathbb{R}[N, k, S, head_dim]$

5) 헤드 연결 (Concatenate Heads)

- 차원 순서 복원 + 병합

$[N, k, S, head_dim]$
→ transpose → $[N, S, k, head_dim]$
→ reshape → $[N, S, k * head_dim] = [N, S, d]$

- 이때 각 헤드의 출력을 임베딩 축으로 이어 붙인 것

6) 최종 선형 투영 (Output Projection)

- 또 하나의 가중치 행렬 $W^O \in \mathbb{R}[d, d]$ 를 적용

$\text{MultiHead}(X) = \text{Concat}(head_1, \dots, head_k) \cdot W^O + b^O$
 $\in \mathbb{R}[N, S, d]$

예시

- $N = 32, S = 100, d = 512, k = 8 \Rightarrow head_dim = 64$

단계	텐서 형태
입력 X	[32, 100, 512]
Q, K, V 투영	[32, 100, 512] × 3
Split & Transpose	[32, 8, 100, 64] × 3
Attention (per head)	[32, 8, 100, 64]
Concat & Reshape	[32, 100, 512]
최종 투영 (Output)	[32, 100, 512]

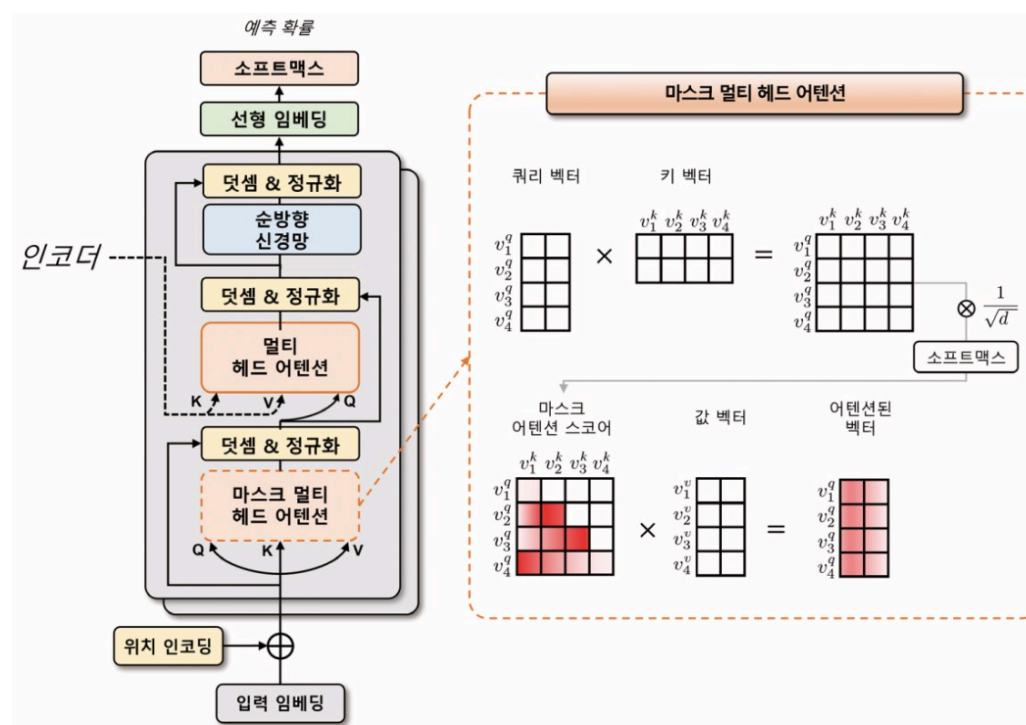
7.6. 트랜스포머 디코더

- 입력: 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩
- 인코더의 멀티 헤드 어텐션 모듈은 **인과성(Casuality)**을 반영한 **마스크 멀티 헤드 어텐션 모듈**로 대체

마스크 멀티 헤드 어텐션

- 어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 바라볼 수 있게 마스크를 씌움
- 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌움
- 셀프 어텐션에서 현재 위치 이전의 단어들만 참조
 - ⇒ 인과성이 보장
- 마스크 영역에서 수치적으로 굉장히 작은 값인 $-\infty$ 마스크를 더해줌
 - ⇒ 해당 영역의 어텐션 스코어값이 0에 가까워짐
 - ⇒ 소프트맥스 계산 시 해당 영역의 어텐션 가중치가 0이 됨
 - ⇒ 마스크 영역 이전에 있는 입력 토큰들에 대해 정보를 참조하지 않게 됨

7.6.1. 트랜스포머 디코더 블록 연산 과정



멀티 헤드 어텐션

- 쿼리 벡터: 타깃 데이터
 - 키 벡터, 값 벡터: 인코더의 소스 데이터
 - ⇒ 쿼리 벡터: 타깃 데이터의 위치 정보를 포함한 입력 임베딩 + 위치 인코딩
1. 쿼리 벡터, 키 벡터, 값 벡터를 이용해 어텐션 스코어 맵 계산
 2. 소프트맥스 함수 적용 → 어텐션 가중치 구함
 3. 어텐션 가중치와 값 벡터를 가중합 → 멀티 헤드 어텐션의 출력 벡터 구함
- 인코더의 멀티 헤드 어텐션과 거의 동일하지만, 디코더가 미래 토큰을 셀프 어텐션을 방지하기 위해 마스크가 적용

다른 트랜스포머 블록

- 여러 개의 트랜스포머 디코더 블록으로 구성
- 이전 트랜스포머 디코더 블록의 산출물은 다음 트랜스포머 디코더 블록의 입력으로 전달
 - ⇒ 이전 시점에서의 정보를 현재 시점에서 활용

선형 변환, 소프트맥스 함수 적용

- 마지막 디코더 블록의 출력 텐서 $[N, S, d]$ 에 선형 변환, 소프트맥스 함수 적용
 - ⇒ 각 타깃 시퀀스 위치마다 예측 확률 계산

7.6.2. 디코더의 입출력 예시

- 디코더는 타겟 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델
- 입력 토큰을 순차적으로 나타내는 방법
 - ⇒ 빈 값을 의미하는 PAD 토큰 사용

예시

- ‘ChatGPT는 트랜스포머 모델로 이뤄져 있다’라는 문장 추론

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

7.7. 모델 실습

[Week12_예습과제_방민지.ipynb](#) [참조](#)

7.6.2. 디코더의 입출력 예시

- 디코더는 타겟 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델
- 입력 토큰을 순차적으로 나타내는 방법
 - ⇒ 빈 값을 의미하는 PAD 토큰 사용

예시

- ‘ChatGPT는 트랜스포머 모델로 이뤄져 있다’라는 문장 추론

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

7.7. 모델 실습

[Week12_예습과제_방민지.ipynb](#) [참조](#)

GPT(Generative Pre-trained Transformer)

7.8. GPT 개념

- 2018년 OpenAI에서 발표한 트랜스포머 기반 언어 모델
- ELMo(Embeddings from Language Model)과 같이 대규모 말뭉치로 사전 학습된 모델
- 다양한 다운스트림 작업에서 우수한 성능을 보임

특징

- 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델
 - 트랜스포머의 인코더: 입력 문장의 특징을 추출하는 데 초점
 - 트랜스포머의 디코더: 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점
- ⇒ 자연어 생성과 같은 언어 모델링 작업에서 더 높은 성능을 발휘
- 대규모 텍스트 데이터세트로 사전 학습해 초기화
- 특정 자연어 처리 작업에 맞게 미세 조정해 사용
- 자연어 생성, 기계 번역, 질의응답, 요약 등의 자연어 처리 작업에 활용
- 예측 성능이 뛰어나고 적은 데이터로도 높은 성능을 보임

7.8.1. GPT 모델 시리즈

- GPT-1(2018), GPT-2(2019), GPT-3(2020)
 - 거의 동일한 트랜스포머 구조 사용
 - 버전이 갱신될수록 더 많은 트랜스포머 계층과 데이터를 활용해 학습됨

GPT-1

- 첫번째 GPT 모델
- 약 1억 2천만개의 매개변수가 존재

GPT-2

- 두번째 발표된 모델
- GPT-1의 성능을 크게 능가
- 약 15억개의 매개변수가 존재

GPT-3

- 1,750억개의 매개변수가 존재
 - ⇒ 초거대 모델
- 매우 뛰어난 문장 생성 성능
- 막대한 양의 매개변수를 가진 모델이기 때문에 문장을 생성하는 데 많은 자원을 필요로 함
 - ⇒ 많은 양의 매개변수를 줄이기 위해 **RLHF(Reinforcement Learning from Human Feedback)** 방법을 도입해 GPT-3.5(InstructGPT) 모델을 학습

GPT-3.5(InstructGPT)

- RLHF(Reinforcement Learning from Human Feedback)
 - 인간의 피드백을 이용하는 강화 학습 방법
 - 인간이 모델이 생성한 결과물을 평가 → 모델이 좋은 결과물을 생성하면 보상을 주어 모델을 학습
 - ⇒ 기존의 학습 데이터만 사용하는 것보다 더욱 정확하고 효율적인 모델 학습이 가능
 - ⇒ 약 60억개의 가중치로도 뛰어난 성능

- ChatGPT 서비스
 - 2022년 10월 송개
 - 대화형 질의응답 시스템
 - GPT-3.5 기반으로 자연스러운 대화 생성
 - 사용자와의 대화를 통해 지속적으로 학습해 발전

GPT-4

- 2023년 3월에 공개
- 이미지 데이터를 함께 학습
- 텍스트 데이터에 국한되지 않고 이미지 데이터도 처리할 수 있게 개선
- 더 복잡한 작업 수행

*GPT-Neo

- GPT 대체 모델
- GPT-3의 구조를 활용한 거대 언어 모델
- 학습, 테스트에 필요한 코드들이 깃허브에 공개되어 있음

*GPT-J

- GPT-3와 유사한 모델
- 학습 데이터와 관련된 라이선스 문제를 해결하기 위해 개발

7.9. GPT-1

- 트랜스포머 구조를 바탕으로 한 단방향(Undirectional) 언어 모델

7.9.1. 특징

단방향 언어 모델

- 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측
- 이전 단어들을 차례대로 입력 & 다음 언어를 예측
- 모델의 계산량이 줄어들어 학습 속도가 빠르며, 모델의 크기가 작아질 수 있음

트랜스포머 구조 활용

- 이전 단어들에 대한 정보로 다음 단어를 예측할 때 트랜스포머 구조 활용
- 디코더 부분만 사용
 - 인코더-디코더 간 어텐션 연산을 제외
 - 모델의 매개변수 수를 줄이면서도 높은 성능을 보임

7.9.2. 트랜스포머 모델 활용

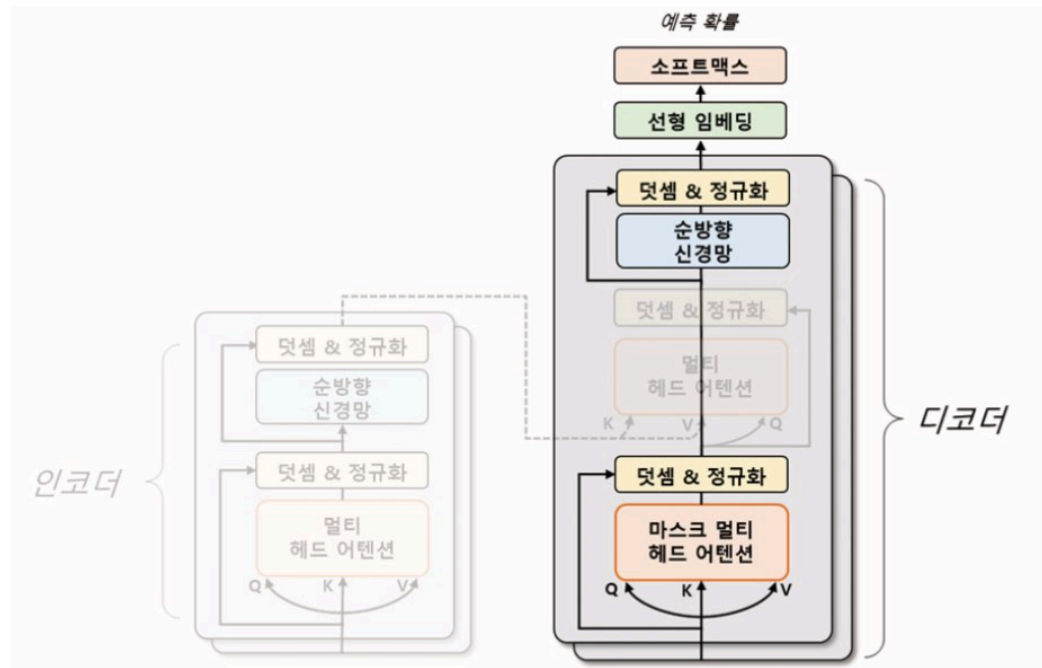


그림 7.7 트랜스포머와 GPT-1 모델 비교

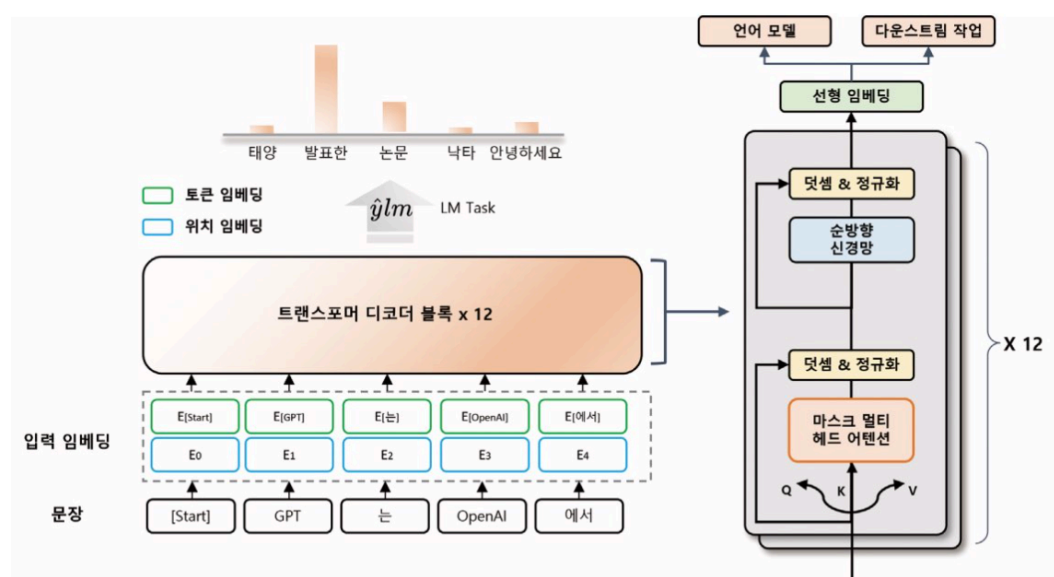
- 인코더를 사용하지 않고 디코더 부분만 사용하여 모델을 구성
- 디코더의 두 번째 멀티 헤드 어텐션은 사용하지 않음

7.9.3. 사전 학습 작업 구조

- 12개의 헤드를 가진 트랜스포머의 디코더를 12층 쌓아 모델을 구성
- 약 1억 2천만 개의 매개변수로 학습
- 4.5GB의 BookCorpus 데이터셋을 사용해 언어 모델을 사전 학습
 - BookCorpus 데이터셋: 출판되지 않은 여러 장르의 책 7,000여 권을 모은 데이터셋

학습 방식

- 입력 문장을 임의의 위치까지 보여주고, 뒤에 이어지는 문장을 모델이 자동으로 예측하게 함
- 별도의 레이블이 필요하지 않기 때문에 비지도 학습으로 학습
 - ⇒ 컴퓨팅 자원만 충분하면 제약 없이 학습 가능



- 입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어 모델을 통해 사전 학습
 1. 입력 문장을 일정한 길이의 블록으로 나누어 처리
 2. 각 블록에서는 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰을 예측하게 함
 - ⇒ 나누어 처리하는 방법은 모델이 장기적인 문맥을 이해할 수 있게 함

입력 임베딩

- 토큰 임베딩 + 위치 임베딩 이용
- 토큰 임베딩: 각 토큰을 고정된 차원의 벡터로 매핑
- 위치 임베딩: 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑

7.9.4. 다운스트림 작업

- 미세 조정을 위한 작업
- 각 작업에 따라 입출력 구조를 바꾸지 않고 언어 모델을 보조 학습해 사용

보조 학습(Auxiliary Learning)

- 모델이 주어진 주요 학습 작업 외에도 다른 작은 부가적인 학습 작업을 동시에 수행하는 것
- 보조 작업에서 학습한 정보를 주요 작업에 전달하여 성능 향상

다운스트림 작업 구조

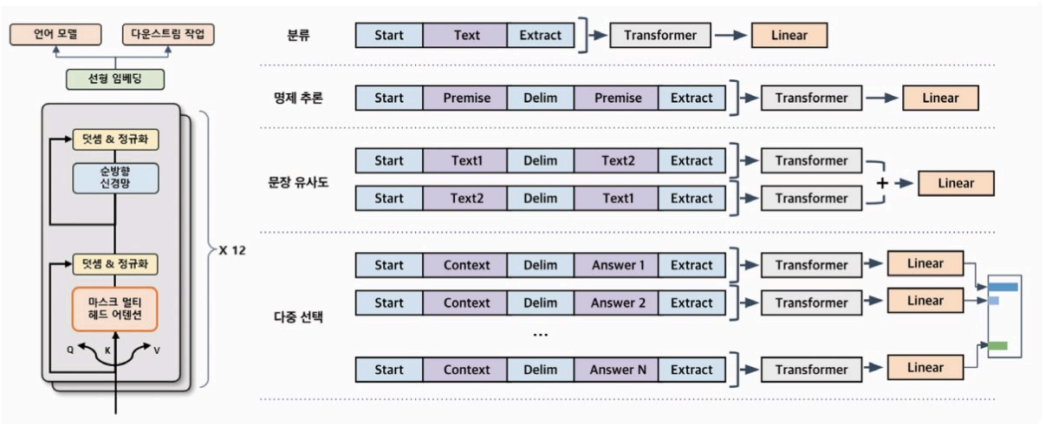


그림 7.9 GPT-1의 다운스트림 작업 구조

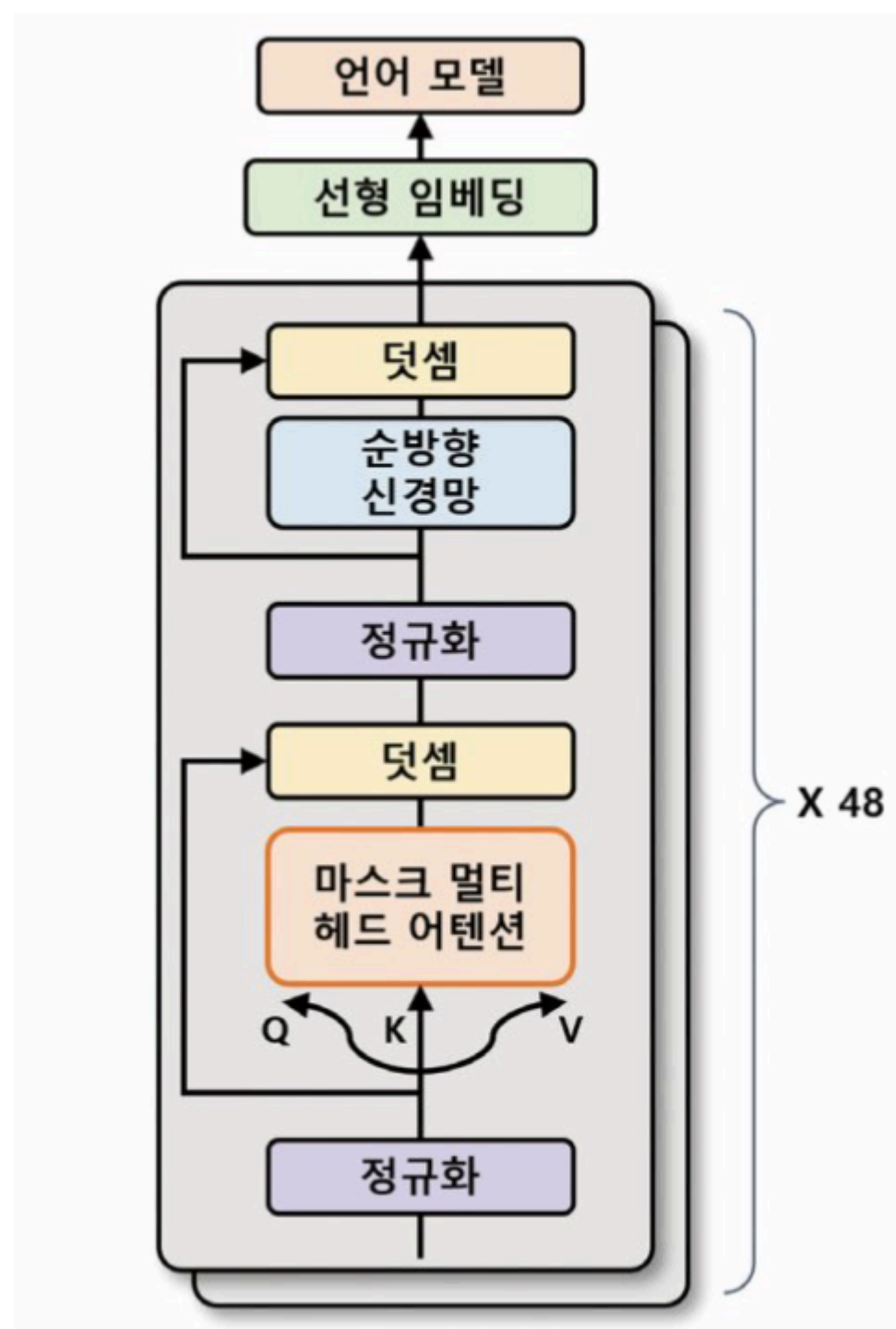
- 원하는 목표에 맞게 모델을 세밀하게 조정해야 함
- 일반적인 언어 모델 학습 시 사용되는 손실 함수 외에 다운스트림 작업에서 필요한 추가 손실 함수가 필요
 - ⇒ **두 개의 손실 함수** 사용해 최적화
 - ⇒ 보조 학습을 통해 언어 모델을 개선 + 다운스트림 작업의 목표를 달성하기 위해 필요한 정보를 학습
- 특수 토큰 사용
 - <start>: 문장의 시작
 - <delim>: 두 문장의 경계
 - <extract>: 문장의 마지막

7.10. GPT-2

7.10.1. GPT-1과의 차이

모델	GPT-1	GPT-2
입력 문장 최대 길이	512	1,024
디코더 계층	12	48
매개변수	1,200만개	15억개

7.10.2. 모델 구조



7.10.2. 사전 학습

- GPT-1보다 훨씬 더 많은 양의 데이터를 이용하여 사전 학습
- 미국의 웹사이트에서 스크래핑한 약 8백만개의 문서를 사용
- 약 40GB 데이터로 사전 학습 수행

7.10.3. 장점

- 더욱 자연스러운 문장 생성
- 더 다양한 분야에서 활용
- 제로-샷 학습 가능
 - 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터 입력
 - 예시) 'GPT는 놀라워요 = GPT is amazing' 같은 형식의 문장 학습
 - '번역 한글 문장 ='을 입력
 - 번역된 영문이 출력
 - 배우지 않은 작업에서도 사용

7.11. GPT-3

7.11.1. 특징

- GPT-2와 모델 구조는 거의 동일
- GPT-2보다 약 116배 많은 매개변수를 가짐

규모가 큰 모델

- 96개의 헤드를 가진 멀티 헤드 어텐션을 사용
- 96개 층의 트랜스포머 디코더 층 사용
- 멀티 헤드 어텐션 과정에서 연산량을 줄이기 위해 희소 어텐션(Sparse Attention)과 일반 어텐션을 섞어 사용

긴 문장 처리

- 토큰 임베딩 크기: 1600(GPT-2) → 12,888
- 최대 입력 토큰 수: 1,024(GPT-2) → 2,048

⇒ 긴 문장에 대한 처리 능력 향상

7.11.2. 사전 학습

- 웹 크롤링, 위키백과, 서적 등에서 수집한 약 45TB에 달하는 방대한 양의 데이터세트 이용
- 다운스트림 작업으로 학습하지 않음
- 질의응답, 번역, 요약, 문서 생성 등 다양한 다운스트림 작업에서도 높은 성능을 보임

7.11.3. 다운스트림 작업

- GPT-2와 같이 프롬프트 형식으로 작업을 수행

다운스트림 작업 프롬프트 예시

질의 응답 입력 프롬프트 문서 : 생성적 사전학습 트랜스포머(Generative Pre-Trained Transformer 3), GPT-3는 OpenAI에서 만든 대형 언어 모델이다. 질문 : GPT-3를 만든 곳은 어디인가? 답변 : OpenAI 출력값	자연어 추론 입력 프롬프트 자연어 추론 : OpenAI는 미국의 인공지능 연구소로, 인류에게 이익을 주는 것을 목표로 한다. GPT-3등 거대 언어 모델을 개발하여 많은 이목을 끌었다. 질문 : OpenAI는 상장사인가? 참, 거짓, 둘 다 아님 답변 : 둘 다 아님 출력값
수학 문제 풀이 입력 프롬프트 질문 : (2 * 4) * 6은 무엇인가? 답변 : 48 출력값	일반 상식 질의 응답 입력 프롬프트 질문 : 물은 섭씨 몇 도에서 어는가? 답변 : 0도 출력값

7.11.4 . 장단점

장점

- 새로운 도메인의 작업에 대해서도 높은 일반화 성능을 보임
- 질의응답, 수학 문제 풀이, 자연어 추론 등과 같은 자연어 처리 분야에서는 기술적 한계를 극복

단점

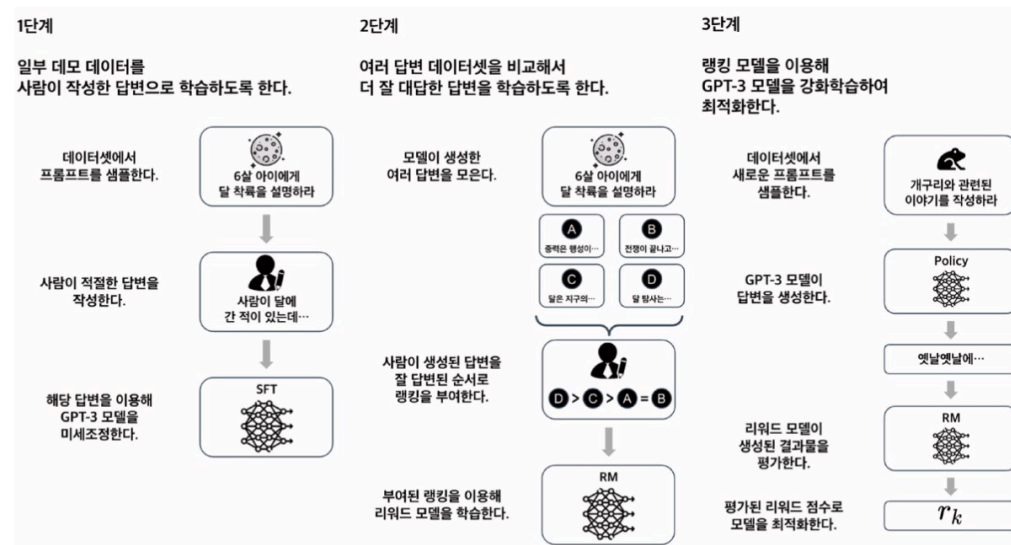
- 큰 모델이기 때문에 매우 느리고 비용이 많이 듦
⇒ 일반 사용자, 소규모 기업에서 활용하기 어려움
- 사전 학습된 데이터에 기반하여 작동
⇒ 데이터가 편향되어 있거나 정확하지 않을 경우 오류가 발생할 수 있음
⇒ 결과물에 대한 검토, 검증 과정이 필요
- 생성된 텍스트의 저작권은 모델을 학습시킨 데이터, 모델을 만든 회사에 있으므로 주의

7.12. GPT-3.5(InstructGPT)

7.12.1. 특징

- GPT-3의 모델 구조를 그대로 따르면서도, 새로운 학습 방법인 RLHF를 도입
- 모델의 매개 변수 수를 줄이고 모델의 자연스러움을 높임

7.12.2. RLHF 학습 방법



지도 미세 조정(Supervised Fine-Tuning, SFT)

1. 데이터셋에서 임의의 프롬프트를 가져옴
 2. 프롬프트에 대해 사람이 직접 적절한 답을 작성
 3. 작성된 답변은 모델이 생성한 문장과 함께 평가
 - 문장이 자연스러우면 보상
 - 문장이 자연스럽지 않으면 보상받지 못함
 4. 보상을 바탕으로 강화 학습을 통해 모델을 학습
- ⇒ GPT-3 모델보다 더 자연스러운 문장 생성

7.12.3. 보상 모델

- 학습 데이터양을 고려하면 전체 프롬프트에 사람이 답변을 다는 것은 현실적으로 어려움
 - ⇒ GPT 모델이 얼마나 잘 답변했는지 평가하는 **보상 모델을 학습**
- 시드(seed)에 따라 다른 답변을 생성하므로 서로 다른 여러 개의 답변이 생성

학습 방법

- 보상 모델로 사람이 평가 → 그에 따른 랭킹을 부여
- 사람이 부여한 랭킹: 생성된 답변의 우수성을 평가한 결과
 - ⇒ 보상 모델의 입력으로 사용
- 보상 모델은 생성된 답변과 랭킹 정보를 활용해 학습
 - ⇒ 모델은 더 자연스러운 문장을 생성

7.12.4. 특징

- RLHF에서는 주어진 프롬프트(state)에 대해 GPT-3 언어 모델(policy)이 답변을 생성(action) → 이를 이용해 사람의 피드백을 기반으로 한 학습된 보상 모델을 평가(reward)
 - ⇒ 이 과정에서 생성된 답변, 랭킹 정보를 활용하여 리워드 모델이 학습
 - ⇒ 자연스러운 문장 생성이 가능해지는 GPT-3.5가 학습
 - ⇒ 이 과정을 반복하여 더욱 정교한 답변 생성 모델을 학습
- 높은 성능 향상을 보였지만 환각 현상 존재
 - 환각 현상(Hallucination): 인간과 같은 추론과 판단 능력을 갖추지 못하고 부적절하거나 오류가 많은 답변을 제시하는 현상

7.13. GPT-4

7.13.1. 특징

- 멀티모달(Multimodal) 모델: 텍스트 데이터뿐만 아니라 이미지 데이터도 인식 가능
- 최대 입력 토큰: 4,096(GPT-3.5) → **32,768**
- 단어 처리: 3,000(GPT-3.5) → **25,000**

7.13.2. 성능

- 대규모 다중 작업 언어 이해(Massive Multitask Language Understanding, MMLU) 벤치마크 테스트
 - GPT-3.5: 70%
 - GPT-4: 85%
- 한국어로 번역한 MMLU 벤치마크
 - GPT-4: 77%
- 미국 변호사 시험
 - GPT-3.5: 213점
 - GPT-4: 298점(상위 10%)
- SAT 읽기, 쓰기 시험
 - GPT-3.5: 670점
 - GPT-4: 710점(상위 7%)

7.14. 모델 실습

[Week12_예습과제_방민지.ipynb](#) 참조